

Module 3 Activity 5: Using SQL in Python

Purpose:

Construct queries to work with feature classes and tables using the **{where_clause}** in tool syntax

NOTE: It is assumed that you are already familiar with proper SQL syntax in ArcGIS.

Instructions:

Any Python environment will work. This activity will build on Activity 2, using search cursors.

Part 1:

1. Import the **arcpy** module
2. Set the workspace
3. Choose a feature class you want to work with
4. Create the **SearchCursor** inside a variable called **SCursor**
5. Reference the feature class in the **SearchCursor** syntax and specify the field you want to search. Fields are a required parameter in the **SearchCursor** and are referenced using square brackets **[]**
6. Use the optional **{where_clause}** parameter to create a query and return values based on that query
7. Create a query that returns values from the **NAME** field that does not equal 'NO CITY'
 - The SQL syntax should be: **STATUS <> 'NO CITY'**
 - Remember that the field delimiters can be different based on feature class type
 - You can always check a SQL syntax in Pro prior to adding it to your script

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME"], "STATUS <> 'NO CITY'")
```

8. The query created in the **{where_clause}** parameter is added
9. Use a print statement that will return the values of a specified field with string formatting to organize your results
10. Run the code and examine your results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME"], "STATUS <> 'NO CITY'")
#start a for loop and return results of the cursor
for row in SCursor:
    #use string formatting to organize your results
    print ("Name: " + row[0])
```

The screenshot below shows the values printed to the screen that satisfies the SQL query specified in the *{where_clause}*

```
Python
Name: HELOTES
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LYTLE
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LIVE OAK
Name: SAINT HEDWIG
Name: KIRBY
Name: CONVERSE
Name: SAN ANTONIO
Name: CHINA GROVE
Name: FAIR OAKS RANCH
Name: OLMOS PARK
Name: BALCONES HEIGHTS
Name: GARDEN RIDGE
Name: BULVERDE
Name: HILL COUNTRY VILLAGE
```

The next steps will use the **AddFieldDelimiters** function to avoid any confusion on which delimiter should be used. Recall from the objective content, that the **AddFieldDelimiters** function automatically recognizes the feature class format.

11. Modify your code from above and add a line below the feature class variable to incorporate the **AddFieldDelimiters** function
12. Create a variable called **delimitedField** that will hold the **AddFieldDelimiters** function
13. The **AddFieldDelimiters** function has two required parameters. Specify the feature class and the field being used in the query

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#use the "AddFieldDelimiters" function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
```

Now that we have a value for the **AddFieldDelimiters** function, it can be used in the **SearchCursor** function so we do not need to worry about the proper delimiter type

14. Modify the SQL query in the **SearchCursor** to reference the **AddFieldDelimiters** function variable

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#use the "AddFieldDelimiters" function
delimitedField = arcpy.AddFieldDelimiters(fc, "STATUS")
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME"], delimitedField + " <> 'NO CITY'")
#start a for loop and return results of the cursor
for row in SCursor:
    #use string formatting to organize your results
    print ("Name: " + row[0])
```

The **delimitedField** variable is used in the query followed by the **+** sign so that it is attached to the remainder of the query. The remainder of the query can be fully enclosed by double quotes and the escape character is no longer needed. The remainder of your script remains the same.

15. Run your script and examine the results

```
Python
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LYTLE
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LIVE OAK
Name: SAINT HEDWIG
Name: KIRBY
Name: CONVERSE
Name: SAN ANTONIO
Name: CHINA GROVE
Name: FAIR OAKS RANCH
Name: OLMOS PARK
Name: BALCONES HEIGHTS
```

Although the **AddFieldDelimiters** function is not required to be incorporated using SQL, it is recommended to use to reduce the chances of errors, especially if you are working with different feature class types.

The **AddFieldDelimiters** function can be incorporated with any function that uses a **{where_clause}**